

```
!pip install yfinance pandas
!pip install mplfinance

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

from google.colab import files

# Core Scientific Computing
import numpy as np
import pandas as pd
import random
import os

# Deep Learning (TensorFlow / Keras)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import load_model

# Machine Learning Utilities
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, r2_score

# Data Source
import yfinance as yf

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
import mplfinance as mpf

# Set Plotly as default backend for pandas
pd.options.plotting.backend = "plotly"

#import visualkeras
```

Task 1: Preparing Data

```
tickers = ['DPZ', 'PZZA', 'MCD']
start_date = '2021-04-01'
end_date = '2026-03-31'

all_stocks = []

for ticker in tickers:
```

```
df = yf.download(ticker, start=start_date, end=end_date,
auto_adjust=True)
```

```
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)
```

```
df = df[['Open', 'High', 'Low', 'Close', 'Volume']].copy()
df.reset_index(inplace=True)
```

```
df['Company'] = ticker
all_stocks.append(df)
```

```
final_df = pd.concat(all_stocks, ignore_index=True)
final_df = final_df.sort_values(by=['Date',
'Company']).reset_index(drop=True)
final_df.to_csv('stocks.csv', index=False)
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```
files.download('stocks.csv')
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```

```
final_df.head(5)
```

```
{"summary":{"name": "final_df", "rows": 3762,
"fields": [{"column": "Date",
"properties": {"dtype": "date", "min":
"2021-04-01 00:00:00", "max": "2026-03-30 00:00:00",
"num_unique_values": 1254, "samples": [
"2026-03-30 00:00:00", "2021-08-25 00:00:00",
"2021-06-15 00:00:00"], "semantic_type": "",
"description": ""}],
"column": "Open", "properties": {"dtype":
"number", "std": 144.69310915650567, "min":
28.9915003551775, "max": 531.0041873302207,
"num_unique_values": 3761, "samples": [
66.98830247057636, 297.1165430438627,
213.424510545741], "semantic_type": "",
"description": ""}],
"column": "High", "properties": {"dtype": "number",
"std": 146.2094070643518, "min": 29.707929597690697,
"max": 534.4789453702589, "num_unique_values": 3760,
"samples": [
301.11406142280407, 213.50529897884246],
"semantic_type": "", "description": ""}],
"column": "Low", "properties": {"
```

```

\ "dtype\ ": \ "number\ ",\n          \ "std\ ": 143.1618454705396,\n
\ "min\ ": 28.81000468870395,\n          \ "max\ ": 524.902008262106,\n
\ "num_unique_values\ ": 3760,\n          \ "samples\ ": [\n
65.70849893111132,\n          296.7646972376234,\n
210.36393135194135\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "Close\ ",\n          \ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 144.68587847130124,\n          \ "min\ ": 29.087024688720703,\n
\ "max\ ": 531.4278564453125,\n          \ "num_unique_values\ ": 3695,\n
\ "samples\ ": [\n          109.70567321777344,\n
35.89745330810547,\n          242.48167419433594\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
          },\n          {\n          \ "column\ ": \ "Volume\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "number\ ",\n          \ "std\ ": 1401882,\n
\ "min\ ": 155000,\n          \ "max\ ": 18792000,\n
\ "num_unique_values\ ": 3444,\n          \ "samples\ ": [\n
2128100,\n          456600,\n          753900\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
          },\n          {\n          \ "column\ ": \ "Company\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "category\ ",\n          \ "num_unique_values\ ":
3,\n          \ "samples\ ": [\n          \ "DPZ\ ",\n          \ "MCD\ ",\n
\ "PZZA\ ",\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          }\n          ]\n
n}", "type": "dataframe", "variable_name": "final_df"}

final_df['Date'] = pd.to_datetime(final_df['Date'])

```

EDA & Data Preprocessing

```

# Example for DPZ
dpz_df = final_df[final_df['Company']=='DPZ'].copy()

dpz_df.describe()

{"summary": "{\n  \ "name\ ": \ "dpz_df\ ",\n  \ "rows\ ": 8,\n  \ "fields\ ":
[\n    {\n      \ "column\ ": \ "Date\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "date\ ",\n      \ "min\ ": \ "1970-01-01
00:00:00.000001254\ ",\n      \ "max\ ": \ "2026-03-30 00:00:00\ ",\n
\ "num_unique_values\ ": 7,\n      \ "samples\ ": [\n
\ "1254\ ",\n      \ "2023-09-28 08:24:06.889952\ ",\n
\ "2024-12-25 12:00:00\ "\n      ],\n      \ "semantic_type\ ": \ "\",\n
      \ "description\ ": \ "\",\n      }\n    },\n    {\n
\ "column\ ": \ "Open\ ",\n      \ "properties\ ": {\n      \ "dtype\ ":
\ "number\ ",\n      \ "std\ ": 347.3269664596724,\n      \ "min\ ":
57.90341632983355,\n      \ "max\ ": 1254.0,\n
\ "num_unique_values\ ": 8,\n      \ "samples\ ": [\n
404.2549229263853,\n      448.0309351722685,\n      1254.0\n      ],\n
      \ "semantic_type\ ": \ "\",\n      \ "description\ ": \ "\",\n
    }\n  }\n}"}

```

```

}\n    },\n    {\n        \"column\": \"High\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 346.72939803580715, \n            \"min\": 58.34620595534906, \n            \"max\": 1254.0, \n            \"num_unique_values\": 8, \n            \"samples\": [\n                409.0959253173963, \n                454.3383047251691, \n                1254.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }, \n        {\n            \"column\": \"Low\", \n            \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 348.1021945012973, \n                \"min\": 57.41427316944395, \n                \"max\": 1254.0, \n                \"num_unique_values\": 8, \n                \"samples\": [\n                    399.35606742148235, \n                    442.4641961245415, \n                    1254.0\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n            }\n        }, \n        {\n            \"column\": \"Close\", \n            \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 347.4495214031016, \n                \"min\": 57.921446006554525, \n                \"max\": 1254.0, \n                \"num_unique_values\": 8, \n                \"samples\": [\n                    404.1778843589187, \n                    448.63075256347656, \n                    1254.0\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n            }, \n            {\n                \"column\": \"Volume\", \n                \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 1230664.393202265, \n                    \"min\": 1254.0, \n                    \"max\": 3821500.0, \n                    \"num_unique_values\": 8, \n                    \"samples\": [\n                        619881.1004784689, \n                        712700.0, \n                        1254.0\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                }\n            }\n        }\n    ], \n    \"type\": \"dataframe\"}

```

```
mcd_df = final_df[final_df['Company'] == 'MCD']
```

```
mcd_df.describe()
```

```

{\"summary\": \"{ \n    \"name\": \"mcd_df\", \n    \"rows\": 8, \n    \"fields\": [\n        {\n            \"column\": \"Date\", \n            \"properties\": {\n                \"dtype\": \"date\", \n                \"min\": \"1970-01-01 00:00:00.000001254\", \n                \"max\": \"2026-03-30 00:00:00\", \n                \"num_unique_values\": 7, \n                \"samples\": [\n                    \"1254\", \n                    \"2023-09-28 08:24:06.889952\", \n                    \"2024-12-25 12:00:00\" \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n            }, \n            {\n                \"column\": \"Open\", \n                \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 372.84713837855475, \n                    \"min\": 32.75448167241798, \n                    \"max\": 1254.0, \n                    \"num_unique_values\": 8, \n                    \"samples\": [\n                        261.01302262618856, \n                        287.5678562621975, \n                        1254.0\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\" \n                }, \n                {\n                    \"column\": \"High\", \n                    \"properties\": {\n                        \"dtype\": \"number\", \n                        \"std\": 372.40875418012894, \n                        \"min\": 33.00995302261119, \n                        \"max\": 1254.0, \n                        \"num_unique_values\": 8, \n                    } \n                } \n            ] \n        } \n    ] \n}

```

```

\"samples\": [\n          262.9439290707429,\n289.9762521997923,\n          1254.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"Low\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 373.34623934510245,\n          \"min\": 32.499761129447144,\n          \"max\": 1254.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n            259.05315128819035,\n            285.2799998556608,\n            1254.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"Close\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 372.83994647140486,\n            \"min\": 32.78328622811994,\n            \"max\": 1254.0,\n            \"num_unique_values\": 8,\n            \"samples\": [\n              261.05930067971946,\n              287.22320556640625,\n              1254.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"Volume\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 6059708.836649671,\n              \"min\": 1254.0,\n              \"max\": 18792000.0,\n              \"num_unique_values\": 8,\n              \"samples\": [\n                3042047.7671451354,\n                3406550.0,\n                1254.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            }\n          }\n        },\n        \"type\": \"dataframe\"

```

```

pizza_df = final_df[final_df['Company'] == 'PZZA']

```

```

pizza_df.describe()

```

```

{"summary": "{\n  \"name\": \"pizza_df\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01 00:00:00.000001254\",\n        \"max\": \"2026-03-30 00:00:00\",\n        \"num_unique_values\": 7,\n        \"samples\": [\n          \"1254\",\n          \"2023-09-28 08:24:06.889952\",\n          \"2024-12-25 12:00:00\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Open\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 422.91498396968177,\n        \"min\": 22.874870606410628,\n        \"max\": 1254.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          65.46128656706477,\n          78.09496897241551,\n          1254.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"High\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 422.56910332100097,\n          \"min\": 23.12178943701013,\n          \"max\": 1254.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n            66.54883954161122,\n            79.66961401044959,\n            1254.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"Low\",\n          \"properties\": {\n

```

```

\ "dtype\ ": \ "number\ ",\n          \ "std\ ": 423.2287039582429,\n
\ "min\ ": 22.651098514648616,\n          \ "max\ ": 1254.0,\n
\ "num_unique_values\ ": 8,\n          \ "samples\ ": [\n
64.34997407094382,\n          77.21445204683718,\n          1254.0\n
],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
}\n      },\n      {\n          \ "column\ ": \ "Close\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "number\ ",\n          \ "std\ ": 422.862833944582,\n
n          \ "min\ ": 22.896121737760833,\n          \ "max\ ": 1254.0,\n
\ "num_unique_values\ ": 8,\n          \ "samples\ ": [\n
65.42783956968803,\n          78.25932884216309,\n          1254.0\n
],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
}\n      },\n      {\n          \ "column\ ": \ "Volume\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "number\ ",\n          \ "std\ ":
4154758.9521984486,\n          \ "min\ ": 1254.0,\n          \ "max\ ":
12264500.0,\n          \ "num_unique_values\ ": 8,\n          \ "samples\ ":
[\n          831350.2392344498,\n          967225.0,\n
1254.0\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n      }\n      ]\n  },"type": "dataframe"}

```

```

pivot_df = final_df.pivot(index='Date', columns='Company',
values='Close')

```

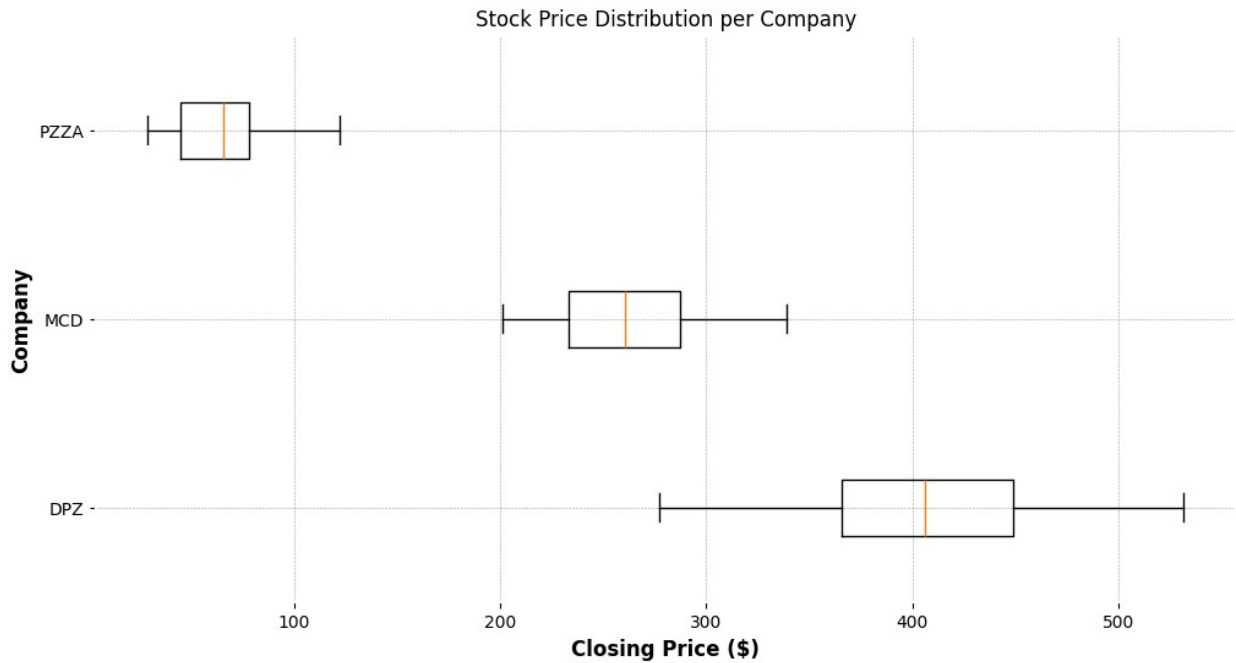
```

plt.figure(figsize=(12,6))
plt.boxplot([pivot_df[col].dropna() for col in pivot_df.columns],
            labels=pivot_df.columns,
            vert=False) # horizontal box plot
plt.title("Stock Price Distribution per Company")
plt.xlabel("Closing Price ($)") # x-axis now shows price
plt.ylabel("Company")
plt.grid(True)
plt.show()

```

/tmp/ipykernel_4923/2612087475.py:2: MatplotlibDeprecationWarning:

The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.



```
corr_matrix = pivot_df.corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation between Companies' Closing Prices")
plt.show()
```



Identifies if scaling or outlier removal is needed before training. Wide price ranges across stocks confirm StandardScaler is necessary to normalize inputs

DPZ Modeling

```
# 30-day rolling mean and std (DPZ only)
rolling_mean = pivot_df['DPZ'].rolling(window=30).mean()
rolling_std = pivot_df['DPZ'].rolling(window=30).std()

plt.figure(figsize=(14,6))

# DPZ actual price
plt.plot(pivot_df['DPZ'], label='DPZ Close')

# rolling mean
plt.plot(rolling_mean, label='30-day Rolling Mean')
```

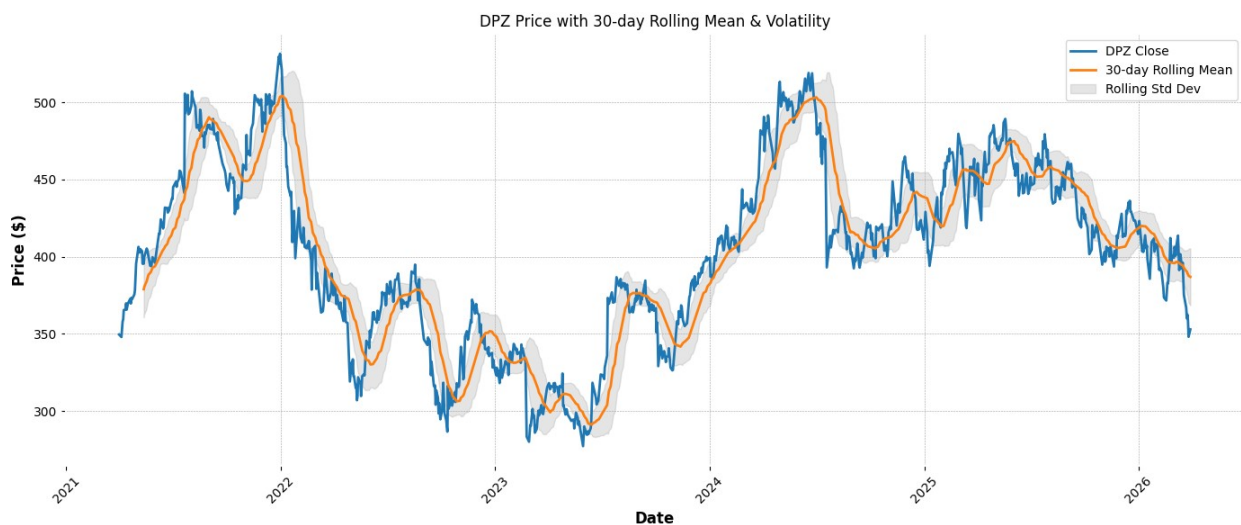


```

# volatility band
plt.fill_between(
    pivot_df.index,
    rolling_mean - rolling_std,
    rolling_mean + rolling_std,
    color='gray',
    alpha=0.2,
    label='Rolling Std Dev'
)

plt.title("DPZ Price with 30-day Rolling Mean & Volatility")
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```

# 1. Convert date
final_df['Date'] = pd.to_datetime(final_df['Date'])

# 2. Set index
df_plot = final_df.set_index('Date')

# 3. Filter ONLY DPZ
ticker = 'DPZ'
ticker_data = df_plot[df_plot['Company'] == ticker].copy()

# 4. Prepare OHLCV data
plot_data = ticker_data[['Open', 'High', 'Low', 'Close', 'Volume']]
plot_data = plot_data.sort_index().dropna()

```

```

# 5. Style
my_style = mpf.make_mpf_style(
    base_mpf_style='charles',
    gridstyle='--',
    facecolor='white'
)

# 6. Plot
fig, axlist = mpf.plot(
    plot_data,
    type='candle',
    style=my_style,
    volume=True,
    ylabel='Price ($)',
    ylabel_lower='Volume',
    figsize=(12, 6),
    returnfig=True,
    tight_layout=True
)

# 7. Title
axlist[0].set_title(
    f"{ticker} Stock Price Over Time",
    fontsize=12,
    pad=12
)

plt.show()

/usr/local/lib/python3.12/dist-packages/mplfinance/
_arg_validators.py:84: UserWarning:

```

```

=====

WARNING: YOU ARE PLOTTING SO MUCH DATA THAT IT MAY NOT BE
        POSSIBLE TO SEE DETAILS (Candles, Ohlc-Bars, Etc.)
For more information see:
- https://github.com/matplotlib/mplfinance/wiki/Plotting-Too-Much-Data

TO SILENCE THIS WARNING, set `type='line'` in `mpf.plot()`
OR set kwarg `warn_too_much_data=N` where N is an integer
LARGER than the number of data points you want to plot.

=====

```



A candlestick chart with volume bars was used to visualize DPZ stock price movements over the five-year period. It shows daily Open, High, Low, and Close prices, helping identify trends, volatility, and price ranges more effectively than a line chart. Volume bars indicate the strength of price movements, while also helping detect support and resistance levels and sudden structural changes that may affect LSTM model performance.

Train Test Data Preparation

```
seed = 42
np.random.seed(seed)
random.seed(seed)
tf.random.set_seed(seed)
os.environ['PYTHONHASHSEED'] = str(seed)
```

Since stock price data is a time series, the observations were first arranged in chronological order to preserve the temporal sequence. An 80:20 train-test split was then applied, where the first 80% of the data was used for training and the remaining 20% for testing. This approach ensures that past data is used to predict future values, maintaining the real-world forecasting structure and preventing data leakage.

The data is split before scaling to prevent data leakage. If scaling is done first, information from the test set influences the scaler, causing unrealistic model performance. By splitting first, the scaler is fitted only on training data and then applied to both training and testing sets, ensuring fair evaluation and realistic forecasting.

```
#DATA PREPARATION
dpz_df_filtered = dpz_df[['Date', 'Close']]
dpz_df_filtered.set_index('Date', inplace=True)
```

```

dpz_df_filtered.index = pd.to_datetime(dpz_df_filtered.index)
dpz_df_filtered = dpz_df_filtered.sort_index()
dpz_train_size = int(len(dpz_df_filtered) * 0.8)
dpz_train_data = dpz_df_filtered[:dpz_train_size]
dpz_test_data = dpz_df_filtered[dpz_train_size:]

dpz_train_data_values = dpz_train_data.values
dpz_test_data_values = dpz_test_data.values

dpz_train_data_values
array([[349.62316895],
       [347.99389648],
       [356.4586792 ],
       ...,
       [459.06317139],
       [463.42922974],
       [439.67166138]])

```

Outlier removal was not applied because sudden price increases or decreases in stock data often represent real market behavior rather than errors. These fluctuations are important patterns influenced by events such as earnings reports, economic news, or market sentiment, and removing them could result in the loss of meaningful information needed for forecasting.

Scaling was necessary because LSTM models perform better when input values are on a similar numerical range. Stock prices can vary significantly in magnitude, and unscaled data may slow training and reduce model performance. By scaling the data, the model learns more efficiently, converges faster, and produces more stable predictions.

```

#SCALING
dpz_scaler = StandardScaler()
dpz_scaled_train_data =
dpz_scaler.fit_transform(dpz_train_data_values)
dpz_scaled_test_data = dpz_scaler.transform(dpz_test_data_values)

# TRAIN DATA (SLIDING WINDOW)
# give sequence_dpz days as input to predict the next day's stock
price
dpz_X_train, dpz_y_train = [], []
sequence_dpz = 90

for i in range(sequence_dpz, len(dpz_scaled_train_data)):
    dpz_X_train.append(dpz_scaled_train_data[i-sequence_dpz:i, 0])
    dpz_y_train.append(dpz_scaled_train_data[i, 0])

dpz_X_train, dpz_y_train = np.array(dpz_X_train),
np.array(dpz_y_train)

```

```

dpz_X_train = np.reshape(dpz_X_train, (dpz_X_train.shape[0],
dpz_X_train.shape[1], 1))

len(dpz_X_train[0])
dpz_y_train[0]

np.float64(1.4555266203281165)

# TEST DATA
dpz_last_points = dpz_scaled_train_data[-sequence_dpz:]
dpz_combined_test_data = np.concatenate((dpz_last_points,
dpz_scaled_test_data), axis=0)

dpz_X_test, dpz_y_test = [], dpz_test_data_values

for i in range(sequence_dpz, len(dpz_combined_test_data)):
    dpz_X_test.append(dpz_combined_test_data[i-sequence_dpz:i, 0])

dpz_X_test = np.array(dpz_X_test)
dpz_X_test = np.reshape(dpz_X_test, (dpz_X_test.shape[0],
dpz_X_test.shape[1], 1))

```

Model Development

LSTM was used because stock price data is sequential and depends on previous observations over time. Unlike traditional machine learning models, LSTM can learn long-term dependencies and temporal patterns in time series data, making it suitable for capturing trends, seasonality, and price movements. Its memory cell structure helps overcome the limitations of standard RNNs, such as the vanishing gradient problem, allowing better performance in stock price forecasting.

```

# MODEL
dpz_model = keras.models.Sequential()

dpz_model.add(keras.layers.LSTM(64, return_sequences=True,
input_shape=(dpz_X_train.shape[1], 1)))
dpz_model.add(keras.layers.LSTM(128, return_sequences=False))
dpz_model.add(keras.layers.Dense(128, activation="relu"))
dpz_model.add(keras.layers.Dropout(0.3))
dpz_model.add(keras.layers.Dense(1))

dpz_model.compile(optimizer="adam",
                  loss="mae",
                  metrics=['mse'])

dpz_model.summary()

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/
rnn.py:199: UserWarning:

```

Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

Model: "sequential_3"

Layer (type) Param #	Output Shape
lstm_6 (LSTM) 16,896	(None, 90, 64)
lstm_7 (LSTM) 98,816	(None, 128)
dense_6 (Dense) 16,512	(None, 128)
dropout_3 (Dropout) 0	(None, 128)
dense_7 (Dense) 129	(None, 1)

Total params: 132,353 (517.00 KB)

Trainable params: 132,353 (517.00 KB)

Non-trainable params: 0 (0.00 B)

```
dpz_first_callback = keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

Training the Model

```
dpz_training = dpz_model.fit(  
    dpz_X_train,
```

```

    dpz_y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    callbacks=[dpz_first_callback],
    shuffle=False
)

```

Epoch 1/50
23/23 _____ 8s 203ms/step - loss: 0.4160 - mse: 0.3263
- val_loss: 0.2254 - val_mse: 0.0824

Epoch 2/50
23/23 _____ 6s 239ms/step - loss: 0.3499 - mse: 0.1909
- val_loss: 0.2212 - val_mse: 0.0856

Epoch 3/50
23/23 _____ 4s 176ms/step - loss: 0.3325 - mse: 0.1731
- val_loss: 0.2206 - val_mse: 0.0858

Epoch 4/50
23/23 _____ 4s 169ms/step - loss: 0.2207 - mse: 0.0888
- val_loss: 0.2115 - val_mse: 0.0821

Epoch 5/50
23/23 _____ 6s 268ms/step - loss: 0.1856 - mse: 0.0596
- val_loss: 0.2039 - val_mse: 0.0727

Epoch 6/50
23/23 _____ 4s 160ms/step - loss: 0.1906 - mse: 0.0649
- val_loss: 0.1900 - val_mse: 0.0607

Epoch 7/50
23/23 _____ 6s 178ms/step - loss: 0.1818 - mse: 0.0611
- val_loss: 0.1868 - val_mse: 0.0609

Epoch 8/50
23/23 _____ 6s 281ms/step - loss: 0.1774 - mse: 0.0529
- val_loss: 0.1854 - val_mse: 0.0593

Epoch 9/50
23/23 _____ 8s 170ms/step - loss: 0.1843 - mse: 0.0562
- val_loss: 0.1801 - val_mse: 0.0574

Epoch 10/50
23/23 _____ 6s 275ms/step - loss: 0.1839 - mse: 0.0558
- val_loss: 0.1768 - val_mse: 0.0550

Epoch 11/50
23/23 _____ 8s 164ms/step - loss: 0.1759 - mse: 0.0517
- val_loss: 0.1747 - val_mse: 0.0534

Epoch 12/50
23/23 _____ 7s 271ms/step - loss: 0.1540 - mse: 0.0409
- val_loss: 0.1719 - val_mse: 0.0532

Epoch 13/50
23/23 _____ 8s 167ms/step - loss: 0.1504 - mse: 0.0410
- val_loss: 0.1830 - val_mse: 0.0605

Epoch 14/50
23/23 _____ 6s 262ms/step - loss: 0.2107 - mse: 0.0860
- val_loss: 0.1865 - val_mse: 0.0588

```
Epoch 15/50
23/23 _____ 4s 162ms/step - loss: 0.1698 - mse: 0.0501
- val_loss: 0.1710 - val_mse: 0.0524
Epoch 16/50
23/23 _____ 4s 162ms/step - loss: 0.1500 - mse: 0.0393
- val_loss: 0.1799 - val_mse: 0.0594
Epoch 17/50
23/23 _____ 5s 206ms/step - loss: 0.1892 - mse: 0.0664
- val_loss: 0.1610 - val_mse: 0.0465
Epoch 18/50
23/23 _____ 5s 215ms/step - loss: 0.1688 - mse: 0.0530
- val_loss: 0.1594 - val_mse: 0.0472
Epoch 19/50
23/23 _____ 4s 168ms/step - loss: 0.1498 - mse: 0.0381
- val_loss: 0.1570 - val_mse: 0.0457
Epoch 20/50
23/23 _____ 4s 162ms/step - loss: 0.1730 - mse: 0.0498
- val_loss: 0.1548 - val_mse: 0.0447
Epoch 21/50
23/23 _____ 6s 219ms/step - loss: 0.1669 - mse: 0.0472
- val_loss: 0.1523 - val_mse: 0.0432
Epoch 22/50
23/23 _____ 4s 163ms/step - loss: 0.1452 - mse: 0.0360
- val_loss: 0.1530 - val_mse: 0.0435
Epoch 23/50
23/23 _____ 4s 163ms/step - loss: 0.1382 - mse: 0.0332
- val_loss: 0.1482 - val_mse: 0.0413
Epoch 24/50
23/23 _____ 6s 264ms/step - loss: 0.1597 - mse: 0.0424
- val_loss: 0.1476 - val_mse: 0.0415
Epoch 25/50
23/23 _____ 4s 163ms/step - loss: 0.1809 - mse: 0.0525
- val_loss: 0.1461 - val_mse: 0.0410
Epoch 26/50
23/23 _____ 5s 165ms/step - loss: 0.1536 - mse: 0.0418
- val_loss: 0.1444 - val_mse: 0.0393
Epoch 27/50
23/23 _____ 7s 224ms/step - loss: 0.1407 - mse: 0.0335
- val_loss: 0.1408 - val_mse: 0.0379
Epoch 28/50
23/23 _____ 4s 159ms/step - loss: 0.1310 - mse: 0.0315
- val_loss: 0.1370 - val_mse: 0.0365
Epoch 29/50
23/23 _____ 4s 162ms/step - loss: 0.1347 - mse: 0.0308
- val_loss: 0.1369 - val_mse: 0.0364
Epoch 30/50
23/23 _____ 7s 230ms/step - loss: 0.1345 - mse: 0.0300
- val_loss: 0.1366 - val_mse: 0.0364
Epoch 31/50
```



```
23/23 _____ 4s 163ms/step - loss: 0.1346 - mse: 0.0310
- val_loss: 0.1316 - val_mse: 0.0342
Epoch 32/50
23/23 _____ 4s 164ms/step - loss: 0.1346 - mse: 0.0310
- val_loss: 0.1305 - val_mse: 0.0338
Epoch 33/50
23/23 _____ 7s 234ms/step - loss: 0.1386 - mse: 0.0333
- val_loss: 0.1303 - val_mse: 0.0339
Epoch 34/50
23/23 _____ 9s 181ms/step - loss: 0.1639 - mse: 0.0438
- val_loss: 0.1271 - val_mse: 0.0324
Epoch 35/50
23/23 _____ 6s 247ms/step - loss: 0.1373 - mse: 0.0318
- val_loss: 0.1283 - val_mse: 0.0332
Epoch 36/50
23/23 _____ 9s 181ms/step - loss: 0.1526 - mse: 0.0377
- val_loss: 0.1268 - val_mse: 0.0325
Epoch 37/50
23/23 _____ 6s 223ms/step - loss: 0.1363 - mse: 0.0330
- val_loss: 0.1234 - val_mse: 0.0306
Epoch 38/50
23/23 _____ 4s 164ms/step - loss: 0.1308 - mse: 0.0297
- val_loss: 0.1233 - val_mse: 0.0308
Epoch 39/50
23/23 _____ 6s 209ms/step - loss: 0.1204 - mse: 0.0270
- val_loss: 0.1228 - val_mse: 0.0308
Epoch 40/50
23/23 _____ 5s 215ms/step - loss: 0.1200 - mse: 0.0259
- val_loss: 0.1199 - val_mse: 0.0291
Epoch 41/50
23/23 _____ 4s 163ms/step - loss: 0.1207 - mse: 0.0254
- val_loss: 0.1223 - val_mse: 0.0301
Epoch 42/50
23/23 _____ 4s 167ms/step - loss: 0.1255 - mse: 0.0267
- val_loss: 0.1184 - val_mse: 0.0283
Epoch 43/50
23/23 _____ 6s 282ms/step - loss: 0.1219 - mse: 0.0269
- val_loss: 0.1187 - val_mse: 0.0283
Epoch 44/50
23/23 _____ 4s 183ms/step - loss: 0.1242 - mse: 0.0263
- val_loss: 0.1183 - val_mse: 0.0283
Epoch 45/50
23/23 _____ 5s 168ms/step - loss: 0.1355 - mse: 0.0317
- val_loss: 0.1187 - val_mse: 0.0286
Epoch 46/50
23/23 _____ 7s 237ms/step - loss: 0.1229 - mse: 0.0264
- val_loss: 0.1182 - val_mse: 0.0283
Epoch 47/50
23/23 _____ 4s 183ms/step - loss: 0.1161 - mse: 0.0244
```

```

- val_loss: 0.1137 - val_mse: 0.0265
Epoch 48/50
23/23 ————— 5s 209ms/step - loss: 0.1240 - mse: 0.0264
- val_loss: 0.1132 - val_mse: 0.0263
Epoch 49/50
23/23 ————— 5s 234ms/step - loss: 0.1243 - mse: 0.0264
- val_loss: 0.1113 - val_mse: 0.0255
Epoch 50/50
23/23 ————— 4s 164ms/step - loss: 0.1341 - mse: 0.0304
- val_loss: 0.1100 - val_mse: 0.0253

# graphical representation
pd.DataFrame(dpz_training.history).plot()

```

Model Evaluation

```

# Make a Prediction
dpz_predictions = dpz_model.predict(dpz_X_test)
dpz_predictions = dpz_scaler.inverse_transform(dpz_predictions)

8/8 ————— 2s 169ms/step

# Plotting data
dpz_test = dpz_test_data
dpz_test['dpz_predictions'] = dpz_predictions
dpz_test.plot()

/tmp/ipykernel_4923/1760518515.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy

# visualize the entire prediction
dpz_fig = go.Figure()

# train data
dpz_fig.add_trace(go.Scatter(x=dpz_train_data.index,
y=dpz_train_data['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue'))))

# test data
dpz_fig.add_trace(go.Scatter(x=dpz_test.index, y=dpz_test['Close'],
mode='lines', name='Test (Actual)', line=dict(color='orange'))))

# predicted data

```

```

dpz_fig.add_trace(go.Scatter(x=dpz_test.index,
y=dpz_test['dpz_predictions'], mode='lines', name='Predictions',
line=dict(color='red')))

# Update the layout
dpz_fig.update_layout(
    title="Our Stock Predictions DPZ",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

dpz_fig.show()

# observe MAE
dpz_mae = mean_absolute_error(dpz_y_test, dpz_predictions)
print(f'Mean Absolute Error (MAE): {dpz_mae}')

Mean Absolute Error (MAE): 5.325133304671938

# goodness of fit
dpz_r2 = r2_score(dpz_y_test, dpz_predictions)
print(f'R-squared (R2) Score: {dpz_r2}')

R-squared (R2) Score: 0.9477793919251926

```

Model Predictions

```

dpz_train_predict = dpz_model.predict(dpz_X_train)
dpz_test_predict = dpz_model.predict(dpz_X_test)

dpz_train_predict = dpz_scaler.inverse_transform(dpz_train_predict)
dpz_test_predict = dpz_scaler.inverse_transform(dpz_test_predict)

29/29 ————— 3s 94ms/step
8/8 ————— 2s 188ms/step

# prepare the full prediction
dpz_full_predictions = pd.DataFrame(np.concatenate((dpz_train_predict,
dpz_test_predict), axis=0))

# Get filtered data starting from index sequence_dpz
dpz_filtered_data_after_seq =
dpz_df_filtered.iloc[sequence_dpz:].copy()
dpz_full_predictions.index = dpz_filtered_data_after_seq.index

# assign column names
dpz_full_predictions.columns = ['Predictions']

# visualize the entire prediction
dpz_fig = go.Figure()

```

```

# full data
dpz_fig.add_trace(go.Scatter(x=dpz_df_filtered.index,
y=dpz_df_filtered['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue')))

# predicted data
dpz_fig.add_trace(go.Scatter(x=dpz_full_predictions.index,
y=dpz_full_predictions['Predictions'], mode='lines',
name='Predictions', line=dict(color='red')))

# Update the layout
dpz_fig.update_layout(
    title="Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

dpz_fig.show()

```

Future Forecast

```

model_path_dpz = "/content/drive/MyDrive/stock_models/dpz_model.keras"
scaler_path_dpz = "/content/drive/MyDrive/stock_models/dpz_scaler.pkl"

model_dpz = load_model(model_path_dpz)

with open(scaler_path_dpz, "rb") as f:
    dpz_scaler_saved = pickle.load(f)

df_dpz_fore = yf.download("DPZ", start="2025-04-01", end="2026-05-01")
/tmp/ipykernel_3310/3246128779.py:3: FutureWarning:
YF.download() has changed argument auto_adjust default to True

[*****100%*****] 1 of 1 completed

# Drop the Ticker level (level 1)
df_dpz_fore.columns = df_dpz_fore.columns.droplevel(1)

# (optional) ensure clean column names
df_dpz_fore.columns.name = None

# Final dataframe
df_dpz_final = df_dpz_fore[['Close', 'High', 'Low', 'Open', 'Volume']]
df_dpz_final = df_dpz_final.reset_index()

df_dpz_filtered = df_dpz_final[['Date', 'Close']]
df_dpz_filtered.set_index('Date', inplace=True)

```

```
dpz_data_values = df_dpz_filtered.values  
dpz_data_values
```

```
array([[451.79629517],  
       [457.71600342],  
       [460.54800415],  
       [456.0541687 ],  
       [436.81015015],  
       [429.29742432],  
       [423.54489136],  
       [448.7479248 ],  
       [445.50289917],  
       [454.47103882],  
       [467.94274902],  
       [460.75454712],  
       [454.54968262],  
       [458.96484375],  
       [460.46932983],  
       [478.04171753],  
       [477.78601074],  
       [479.84115601],  
       [479.45767212],  
       [482.46673584],  
       [485.21020508],  
       [482.20120239],  
       [473.37084961],  
       [476.53713989],  
       [469.87991333],  
       [471.50247192],  
       [468.73925781],  
       [468.87692261],  
       [470.92230225],  
       [474.07876587],  
       [476.33065796],  
       [475.18014526],  
       [477.81549072],  
       [486.80322266],  
       [489.23208618],  
       [484.85620117],  
       [472.48577881],  
       [474.22631836],  
       [472.42678833],  
       [476.48800659],  
       [475.06210327],  
       [466.61523438],  
       [465.9269104 ],  
       [458.23718262],  
       [456.79165649],  
       [458.14868164],  
       [452.36663818],
```

[460.41036987],
[460.1546936],
[447.90222168],
[439.45535278],
[443.48703003],
[446.33990479],
[448.61038208],
[452.88470459],
[441.02905273],
[441.53250122],
[451.72976685],
[454.05944824],
[447.30731201],
[448.87689209],
[440.65393066],
[444.80984497],
[451.65078735],
[448.10690308],
[446.36950684],
[447.3961792],
[453.52636719],
[457.52432251],
[463.8815918],
[458.51147461],
[466.86276245],
[459.40979004],
[459.32089233],
[462.68710327],
[459.96258545],
[456.30023193],
[474.7895813],
[471.90710449],
[470.21908569],
[479.29098511],
[468.22503662],
[464.0098877],
[469.27142334],
[457.2578125],
[460.17977905],
[461.96652222],
[441.38446045],
[438.07745361],
[436.73495483],
[434.38552856],
[434.87911987],
[435.52075195],
[445.18496704],
[444.79995728],
[445.46133423],

[439.13372803],
[444.77035522],
[437.16928101],
[441.11785889],
[445.19482422],
[454.74053955],
[445.42184448],
[446.73477173],
[443.19088745],
[452.41085815],
[461.29522705],
[460.60421753],
[456.65563965],
[457.86978149],
[460.12051392],
[453.07229614],
[444.77035522],
[450.75247192],
[446.24118042],
[445.88446045],
[438.42251587],
[434.2505188],
[426.19396973],
[424.83636475],
[421.12023926],
[420.05996704],
[421.45715332],
[418.9302063],
[431.81277466],
[430.48486328],
[427.80926514],
[424.2913208],
[427.34350586],
[422.85443115],
[419.08877563],
[407.7124939],
[409.76382446],
[401.66760254],
[402.69824219],
[404.57113647],
[420.3968811],
[413.68798828],
[413.18261719],
[412.49887085],
[419.11849976],
[416.08615112],
[420.98150635],
[417.94915771],
[411.11151123],

[408.97103882],
[407.25665283],
[402.20272827],
[400.30999756],
[394.85968018],
[399.98300171],
[403.2036438],
[396.79208374],
[397.83258057],
[406.4737854],
[403.90719604],
[405.53240967],
[400.11184692],
[404.98733521],
[406.45401001],
[400.97393799],
[399.69561768],
[396.05877686],
[393.62097168],
[403.71890259],
[399.03167725],
[413.84655762],
[411.8348999],
[415.83840942],
[420.02029419],
[420.80316162],
[427.78945923],
[423.21121216],
[412.78622437],
[414.68890381],
[413.42047119],
[415.09521484],
[429.7515564],
[427.82910156],
[435.30133057],
[431.71942139],
[436.21673584],
[429.20214844],
[429.34146118],
[425.73965454],
[424.03820801],
[423.29199219],
[423.4213562],
[419.13299561],
[419.7996521],
[414.72528076],
[423.14276123],
[409.37231445],
[404.4571228],

[403.4621582],
[404.53671265],
[403.05419922],
[413.85964966],
[408.8449707],
[405.33270264],
[407.03411865],
[398.2684021],
[385.69192505],
[396.91525269],
[403.29299927],
[409.43200684],
[411.82989502],
[411.7901001],
[403.33279419],
[405.03421021],
[408.2678833],
[407.16345215],
[399.09420776],
[399.71112061],
[391.45281982],
[392.89553833],
[390.18920898],
[387.77142334],
[382.07019043],
[379.86135864],
[373.61291504],
[371.62298584],
[381.0652771],
[383.19451904],
[382.67712402],
[398.34796143],
[412.11843872],
[401.91000366],
[397.28338623],
[400.48718262],
[399.35290527],
[406.32769775],
[404.57653809],
[400.67623901],
[406.35754395],
[413.69049072],
[398.50717163],
[391.3135376],
[393.99002075],
[401.63000488],
[392.26998901],
[396.17999268],
[376.92001343],

```
[375.29998779],
[373.3500061 ],
[366.79998779],
[360.08999634],
[362.52999878],
[358.54000854],
[348.14001465],
[352.94000244],
[358.79000854],
[361.54000854],
[370.82000732],
[380.76998901],
[368.1499939 ],
[376.23999023],
[373.29998779],
[365.95001221],
[368.82998657],
[369.17001343],
[368.07000732],
[369.      ],
[372.05999756],
[369.45999146],
[369.20001221],
[365.92999268],
[372.39001465],
[367.82998657],
[335.29998779],
[340.45999146],
[332.30999756],
[338.05999756]])
```

```
dpz_scaled_forecast_data = dpz_scaler_saved.transform(dpz_data_values)
```

```
# Take the last sequence_dpz days from the dataset for generating  
future predictions
```

```
dpz_last_days = dpz_scaled_forecast_data[-sequence_dpz:]
```

```
# Reshape
```

```
dpz_x_future = dpz_last_days.reshape((1, dpz_last_days.shape[0], 1))
```

```
# to store predictions for the next 15 days
```

```
dpz_future_predictions = []
```

```
for _ in range(15):
```

```
    dpz_pred = model_dpz.predict(dpz_x_future)
```

```
    dpz_future_predictions.append(dpz_pred[0, 0])
```

```
    dpz_x_future = np.append(dpz_x_future[:, 1:, :], [[dpz_pred[0]]],  
axis=1)
```

```

1/1 _____ 1s 512ms/step
1/1 _____ 0s 89ms/step
1/1 _____ 0s 91ms/step
1/1 _____ 0s 124ms/step
1/1 _____ 0s 92ms/step
1/1 _____ 0s 115ms/step
1/1 _____ 0s 106ms/step
1/1 _____ 0s 91ms/step
1/1 _____ 0s 88ms/step
1/1 _____ 0s 91ms/step
1/1 _____ 0s 82ms/step
1/1 _____ 0s 79ms/step
1/1 _____ 0s 88ms/step
1/1 _____ 0s 91ms/step
1/1 _____ 0s 132ms/step

```

Convert back to the original scale

```

dpz_future_predictions =
dpz_scaler.inverse_transform(np.array(dpz_future_predictions).reshape(
-1, 1))
dpz_future_predictions

```

```

array([[342.2401 ],
       [345.67004],
       [349.80933],
       [354.13263],
       [358.36993],
       [362.3719 ],
       [366.059  ],
       [369.39578],
       [372.37012],
       [374.9682 ],
       [377.24146],
       [379.19217],
       [380.8526 ],
       [382.27567],
       [383.49164]], dtype=float32)

```

Store in a df

```

dpz_forecast_dates = pd.date_range(start=df_dpz_filtered.index[-1] +
pd.Timedelta(days=1), periods=15, freq='B')
dpz_forecast = pd.DataFrame(dpz_future_predictions,
index=dpz_forecast_dates, columns=['Prediction'])

```

dpz_forecast

```

{"summary":{"\n  \"name\": \"dpz_forecast\", \n  \"rows\": 15, \n  \"fields\": [\n    {\n      \"column\": \"Prediction\", \n      \"properties\": {\n        \"dtype\": \"float32\", \n        \"num_unique_values\": 15, \n        \"samples\": [\n

```

```

374.96820068359375,\n          379.1921691894531,\n
342.2401123046875\n          ],\n          \"semantic_type\": \"\", \n
\"description\": \"\" \n          } \n          ] \n
n} \", \"type\": \"dataframe\", \"variable_name\": \"dpz_forecast\"}

# visualize the prediction
dpz_fig = go.Figure()

# test data
dpz_fig.add_trace(go.Scatter(x=df_dpz_filtered.index,
y=df_dpz_filtered['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue'))

# future data
dpz_fig.add_trace(go.Scatter(x=dpz_forecast.index,
y=dpz_forecast['Prediction'], mode='lines', name='Predictions',
line=dict(color='red'))

# Update the layout
dpz_fig.update_layout(
    title="DPZ Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

dpz_fig.show()

```

PZZA Modeling

```

# 30-day rolling mean and std
rolling_mean = pivot_df['PZZA'].rolling(window=30).mean()
rolling_std = pivot_df['PZZA'].rolling(window=30).std()

plt.figure(figsize=(14,6))

# PZZA actual price
plt.plot(pivot_df['PZZA'], label='PZZA Close')

# rolling mean
plt.plot(rolling_mean, label='30-day Rolling Mean')

# volatility band
plt.fill_between(
    pivot_df.index,
    rolling_mean - rolling_std,
    rolling_mean + rolling_std,
    color='gray',
    alpha=0.2,

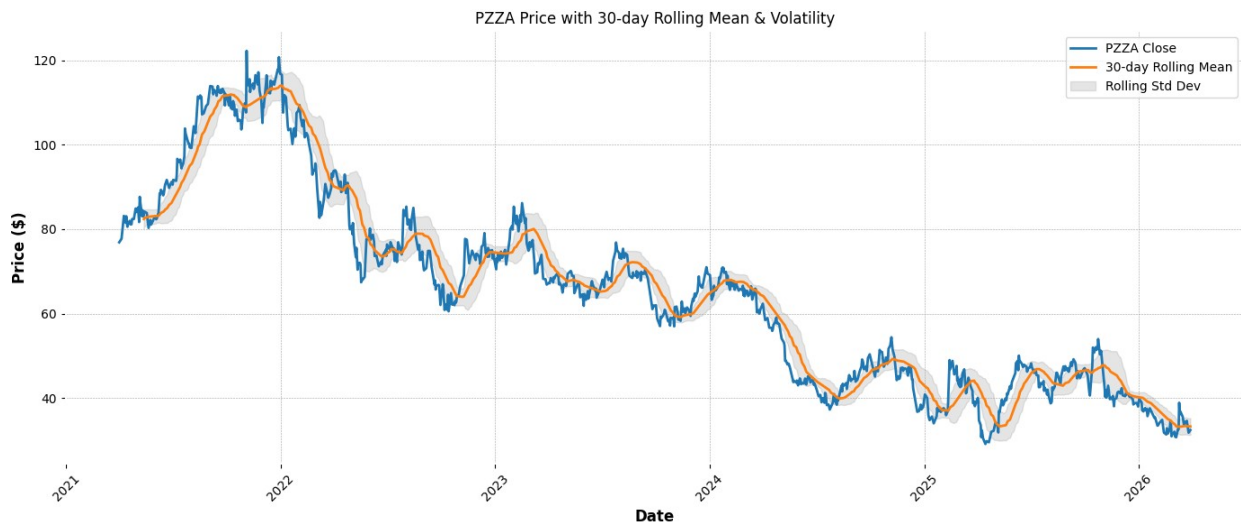
```

```

        label='Rolling Std Dev'
    )

plt.title("PZZA Price with 30-day Rolling Mean & Volatility")
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```

# 1. Convert date
final_df['Date'] = pd.to_datetime(final_df['Date'])

# 2. Set index
df_plot = final_df.set_index('Date')

ticker = 'PZZA'
ticker_data = df_plot[df_plot['Company'] == ticker].copy()

# 4. Prepare OHLCV data
plot_data = ticker_data[['Open', 'High', 'Low', 'Close', 'Volume']]
plot_data = plot_data.sort_index().dropna()

# 5. Style
my_style = mpf.make_mpf_style(
    base_mpf_style='charles',
    gridstyle='--',
    facecolor='white'
)

# 6. Plot

```

```

fig, axlist = mpf.plot(
    plot_data,
    type='candle',
    style=my_style,
    volume=True,
    ylabel='Price ($)',
    ylabel_lower='Volume',
    figsize=(12, 6),
    returnfig=True,
    tight_layout=True
)

# 7. Title
axlist[0].set_title(
    f"{ticker} Stock Price Over Time",
    fontsize=12,
    pad=12
)

plt.show()

/usr/local/lib/python3.12/dist-packages/mplfinance/
_arg_validators.py:84: UserWarning:

```

```

=====

WARNING: YOU ARE PLOTTING SO MUCH DATA THAT IT MAY NOT BE
        POSSIBLE TO SEE DETAILS (Candles, Ohlc-Bars, Etc.)
For more information see:
- https://github.com/matplotlib/mplfinance/wiki/Plotting-Too-Much-Data

TO SILENCE THIS WARNING, set `type='line'` in `mpf.plot()`
OR set kwarg `warn_too_much_data=N` where N is an integer
LARGER than the number of data points you want to plot.

=====

```



Train Test Data Preparation

```
seed = 42
np.random.seed(seed)

random.seed(seed)
tf.random.set_seed(seed)
os.environ['PYTHONHASHSEED'] = str(seed)

# DATA PREPARATION
pzza_df_filtered = pzza_df[['Date', 'Close']]
pzza_df_filtered.set_index('Date', inplace=True)

pzza_df_filtered.index = pd.to_datetime(pzza_df_filtered.index)
pzza_df_filtered = pzza_df_filtered.sort_index()

pzza_train_size = int(len(pzza_df_filtered) * 0.8)

pzza_train_data = pzza_df_filtered[:pzza_train_size]
pzza_test_data = pzza_df_filtered[pzza_train_size:]

pzza_train_data_values = pzza_train_data.values
pzza_test_data_values = pzza_test_data.values

# SCALING
pzza_scaler = StandardScaler()
pzza_scaled_train_data =
pzza_scaler.fit_transform(pzza_train_data_values)
pzza_scaled_test_data = pzza_scaler.transform(pzza_test_data_values)
```

```

# SEQUENCE
sequence_pzza = 90

# TRAIN DATA
pzza_X_train, pzza_y_train = [], []

for i in range(sequence_pzza, len(pzza_scaled_train_data)):
    pzza_X_train.append(pzza_scaled_train_data[i-sequence_pzza:i, 0])
    pzza_y_train.append(pzza_scaled_train_data[i, 0])

pzza_X_train, pzza_y_train = np.array(pzza_X_train),
np.array(pzza_y_train)

pzza_X_train = np.reshape(pzza_X_train, (pzza_X_train.shape[0],
pzza_X_train.shape[1], 1))

# TEST DATA
pzza_last_points = pzza_scaled_train_data[-sequence_pzza:]
pzza_combined_test_data = np.concatenate((pzza_last_points,
pzza_scaled_test_data), axis=0)

pzza_X_test, pzza_y_test = [], pzza_test_data_values

for i in range(sequence_pzza, len(pzza_combined_test_data)):
    pzza_X_test.append(pzza_combined_test_data[i-sequence_pzza:i, 0])

pzza_X_test = np.array(pzza_X_test)
pzza_X_test = np.reshape(pzza_X_test, (pzza_X_test.shape[0],
pzza_X_test.shape[1], 1))

```

Model Development

```

# MODEL
pzza_model = keras.models.Sequential()

pzza_model.add(keras.layers.LSTM(256, return_sequences=True,
input_shape=(pzza_X_train.shape[1], 1)))
pzza_model.add(keras.layers.LSTM(128, return_sequences=False))
pzza_model.add(keras.layers.Dense(128, activation="relu"))
pzza_model.add(keras.layers.Dropout(0.3))
pzza_model.add(keras.layers.Dense(1))

pzza_model.compile(optimizer="adam",
                    loss="mae",
                    metrics=['mse'])
pzza_model.summary()

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/
rnn.py:199: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When

```


using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

Model: "sequential_4"

Layer (type) Param #	Output Shape	
lstm_8 (LSTM) 264,192	(None, 90, 256)	
lstm_9 (LSTM) 197,120	(None, 128)	
dense_8 (Dense) 16,512	(None, 128)	
dropout_4 (Dropout) 0	(None, 128)	
dense_9 (Dense) 129	(None, 1)	

Total params: 477,953 (1.82 MB)

Trainable params: 477,953 (1.82 MB)

Non-trainable params: 0 (0.00 B)

```
pizza_first_callback = keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

Training the Model

```
# TRAINING  
pizza_training = pizza_model.fit(  
    pizza_X_train,  
    pizza_y_train,
```

```
epochs=50,  
batch_size=32,  
validation_split=0.2,  
callbacks=[pzza_first_callback],  
shuffle=False  
)  
  
Epoch 1/50  
23/23 _____ 20s 706ms/step - loss: 0.4069 - mse: 0.4398  
- val_loss: 0.1216 - val_mse: 0.0231  
Epoch 2/50  
23/23 _____ 12s 539ms/step - loss: 0.3783 - mse: 0.3183  
- val_loss: 0.2306 - val_mse: 0.0685  
Epoch 3/50  
23/23 _____ 12s 532ms/step - loss: 0.1888 - mse: 0.0609  
- val_loss: 0.1298 - val_mse: 0.0261  
Epoch 4/50  
23/23 _____ 22s 576ms/step - loss: 0.2698 - mse: 0.1541  
- val_loss: 0.2132 - val_mse: 0.0575  
Epoch 5/50  
23/23 _____ 17s 756ms/step - loss: 0.2279 - mse: 0.0951  
- val_loss: 0.1238 - val_mse: 0.0235  
Epoch 6/50  
23/23 _____ 13s 541ms/step - loss: 0.1687 - mse: 0.0526  
- val_loss: 0.0961 - val_mse: 0.0153  
Epoch 7/50  
23/23 _____ 20s 537ms/step - loss: 0.2550 - mse: 0.1165  
- val_loss: 0.2609 - val_mse: 0.0787  
Epoch 8/50  
23/23 _____ 20s 537ms/step - loss: 0.1294 - mse: 0.0321  
- val_loss: 0.0812 - val_mse: 0.0114  
Epoch 9/50  
23/23 _____ 20s 498ms/step - loss: 0.1978 - mse: 0.0728  
- val_loss: 0.1593 - val_mse: 0.0330  
Epoch 10/50  
23/23 _____ 21s 527ms/step - loss: 0.1152 - mse: 0.0238  
- val_loss: 0.0906 - val_mse: 0.0136  
Epoch 11/50  
23/23 _____ 12s 529ms/step - loss: 0.1615 - mse: 0.0543  
- val_loss: 0.0967 - val_mse: 0.0144  
Epoch 12/50  
23/23 _____ 20s 520ms/step - loss: 0.1165 - mse: 0.0249  
- val_loss: 0.1116 - val_mse: 0.0193  
Epoch 13/50  
23/23 _____ 12s 526ms/step - loss: 0.1312 - mse: 0.0315  
- val_loss: 0.0763 - val_mse: 0.0100  
Epoch 14/50  
23/23 _____ 12s 524ms/step - loss: 0.1214 - mse: 0.0285  
- val_loss: 0.0758 - val_mse: 0.0098  
Epoch 15/50
```

```
23/23 _____ 10s 423ms/step - loss: 0.1694 - mse: 0.0611
- val_loss: 0.0892 - val_mse: 0.0123
Epoch 16/50
23/23 _____ 12s 512ms/step - loss: 0.1289 - mse: 0.0330
- val_loss: 0.0779 - val_mse: 0.0103
Epoch 17/50
23/23 _____ 21s 560ms/step - loss: 0.1057 - mse: 0.0205
- val_loss: 0.0726 - val_mse: 0.0091
Epoch 18/50
23/23 _____ 10s 450ms/step - loss: 0.1186 - mse: 0.0261
- val_loss: 0.0705 - val_mse: 0.0086
Epoch 19/50
23/23 _____ 22s 535ms/step - loss: 0.1285 - mse: 0.0298
- val_loss: 0.0779 - val_mse: 0.0097
Epoch 20/50
23/23 _____ 18s 428ms/step - loss: 0.1468 - mse: 0.0395
- val_loss: 0.0719 - val_mse: 0.0087
Epoch 21/50
23/23 _____ 13s 525ms/step - loss: 0.1060 - mse: 0.0210
- val_loss: 0.0819 - val_mse: 0.0108
Epoch 22/50
23/23 _____ 13s 553ms/step - loss: 0.1219 - mse: 0.0302
- val_loss: 0.0680 - val_mse: 0.0080
Epoch 23/50
23/23 _____ 20s 536ms/step - loss: 0.1029 - mse: 0.0196
- val_loss: 0.0703 - val_mse: 0.0084
Epoch 24/50
23/23 _____ 20s 533ms/step - loss: 0.1016 - mse: 0.0197
- val_loss: 0.0780 - val_mse: 0.0103
Epoch 25/50
23/23 _____ 13s 554ms/step - loss: 0.1097 - mse: 0.0220
- val_loss: 0.0647 - val_mse: 0.0071
Epoch 26/50
23/23 _____ 20s 519ms/step - loss: 0.1005 - mse: 0.0186
- val_loss: 0.0633 - val_mse: 0.0067
Epoch 27/50
23/23 _____ 20s 495ms/step - loss: 0.1578 - mse: 0.0503
- val_loss: 0.0646 - val_mse: 0.0071
Epoch 28/50
23/23 _____ 11s 463ms/step - loss: 0.1120 - mse: 0.0254
- val_loss: 0.0776 - val_mse: 0.0098
Epoch 29/50
23/23 _____ 13s 551ms/step - loss: 0.1011 - mse: 0.0198
- val_loss: 0.0684 - val_mse: 0.0071
Epoch 30/50
23/23 _____ 19s 470ms/step - loss: 0.1109 - mse: 0.0214
- val_loss: 0.0629 - val_mse: 0.0068
Epoch 31/50
23/23 _____ 22s 529ms/step - loss: 0.0987 - mse: 0.0198
```

```
- val_loss: 0.0618 - val_mse: 0.0065
Epoch 32/50
23/23 _____ 18s 437ms/step - loss: 0.1433 - mse: 0.0446
- val_loss: 0.0868 - val_mse: 0.0109
Epoch 33/50
23/23 _____ 11s 488ms/step - loss: 0.0969 - mse: 0.0181
- val_loss: 0.0778 - val_mse: 0.0100
Epoch 34/50
23/23 _____ 12s 521ms/step - loss: 0.1135 - mse: 0.0257
- val_loss: 0.0993 - val_mse: 0.0145
Epoch 35/50
23/23 _____ 12s 522ms/step - loss: 0.1131 - mse: 0.0244
- val_loss: 0.0592 - val_mse: 0.0058
Epoch 36/50
23/23 _____ 10s 434ms/step - loss: 0.0993 - mse: 0.0175
- val_loss: 0.0581 - val_mse: 0.0057
Epoch 37/50
23/23 _____ 12s 494ms/step - loss: 0.1014 - mse: 0.0205
- val_loss: 0.0680 - val_mse: 0.0071
Epoch 38/50
23/23 _____ 13s 552ms/step - loss: 0.0875 - mse: 0.0149
- val_loss: 0.0622 - val_mse: 0.0067
Epoch 39/50
23/23 _____ 12s 527ms/step - loss: 0.0881 - mse: 0.0151
- val_loss: 0.0566 - val_mse: 0.0053
Epoch 40/50
23/23 _____ 11s 463ms/step - loss: 0.0963 - mse: 0.0171
- val_loss: 0.0541 - val_mse: 0.0051
Epoch 41/50
23/23 _____ 11s 472ms/step - loss: 0.1080 - mse: 0.0206
- val_loss: 0.0756 - val_mse: 0.0092
Epoch 42/50
23/23 _____ 12s 532ms/step - loss: 0.0888 - mse: 0.0144
- val_loss: 0.0533 - val_mse: 0.0048
Epoch 43/50
23/23 _____ 12s 526ms/step - loss: 0.1113 - mse: 0.0247
- val_loss: 0.0564 - val_mse: 0.0053
Epoch 44/50
23/23 _____ 12s 524ms/step - loss: 0.0945 - mse: 0.0171
- val_loss: 0.0536 - val_mse: 0.0049
Epoch 45/50
23/23 _____ 10s 420ms/step - loss: 0.0826 - mse: 0.0136
- val_loss: 0.0526 - val_mse: 0.0048
Epoch 46/50
23/23 _____ 12s 519ms/step - loss: 0.0842 - mse: 0.0135
- val_loss: 0.0524 - val_mse: 0.0048
Epoch 47/50
23/23 _____ 20s 500ms/step - loss: 0.0915 - mse: 0.0159
- val_loss: 0.0522 - val_mse: 0.0047
```

```

Epoch 48/50
23/23 ————— 11s 461ms/step - loss: 0.0905 - mse: 0.0167
- val_loss: 0.0579 - val_mse: 0.0059
Epoch 49/50
23/23 ————— 12s 525ms/step - loss: 0.1329 - mse: 0.0333
- val_loss: 0.0575 - val_mse: 0.0054
Epoch 50/50
23/23 ————— 13s 553ms/step - loss: 0.0848 - mse: 0.0131
- val_loss: 0.0540 - val_mse: 0.0049

# PLOTTING
pd.DataFrame(pzza_training.history).plot()

```

Model Evaluation

```

# PREDICTION
pzza_predictions = pzza_model.predict(pzza_X_test)
pzza_predictions = pzza_scaler.inverse_transform(pzza_predictions)

8/8 ————— 2s 215ms/step

pzza_test = pzza_test_data
pzza_test['pzza_predictions'] = pzza_predictions
pzza_test.plot()

/tmp/ipykernel_3310/3955888935.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy

pzza_fig = go.Figure()

pzza_fig.add_trace(go.Scatter(x=pzza_train_data.index,
y=pzza_train_data['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue')))
pzza_fig.add_trace(go.Scatter(x=pzza_test.index, y=pzza_test['Close'],
mode='lines', name='Test (Actual)', line=dict(color='orange')))
pzza_fig.add_trace(go.Scatter(x=pzza_test.index,
y=pzza_test['pzza_predictions'], mode='lines', name='Predictions',
line=dict(color='red')))

pzza_fig.update_layout(
    title="Stock Predictions PZZA",
    xaxis_title="Date",
    yaxis_title="Close Price"

```

```

)

pizza_fig.show()

# MAE
pizza_mae = mean_absolute_error(pzza_y_test, pzza_predictions)
print(f'MAE: {pizza_mae}')

MAE: 0.9783345104688668

# R2
pizza_r2 = r2_score(pzza_y_test, pzza_predictions)
print(f'R²: {pizza_r2}')

R²: 0.9441585454404999

```

Model Predictions

```

pizza_train_predict = pzza_model.predict(pzza_X_train)
pizza_test_predict = pzza_model.predict(pzza_X_test)

pizza_train_predict = pzza_scaler.inverse_transform(pzza_train_predict)
pizza_test_predict = pzza_scaler.inverse_transform(pzza_test_predict)

29/29 ————— 3s 120ms/step
8/8 ————— 1s 122ms/step

pizza_full_predictions =
pd.DataFrame(np.concatenate((pizza_train_predict, pizza_test_predict),
axis=0))

pizza_filtered_data_after_seq =
pizza_df_filtered.iloc[sequence_pzza:].copy()
pizza_full_predictions.index = pizza_filtered_data_after_seq.index

pizza_full_predictions.columns = ['Predictions']

pizza_fig = go.Figure()

pizza_fig.add_trace(go.Scatter(x=pzza_df_filtered.index,
y=pzza_df_filtered['Close'], mode='lines', name='Actual',
line=dict(color='blue')))
pizza_fig.add_trace(go.Scatter(x=pzza_full_predictions.index,
y=pzza_full_predictions['Predictions'], mode='lines',
name='Predictions', line=dict(color='red')))

pizza_fig.update_layout(
    title="Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

```

```
pzza_fig.show()
```

Future Forecast

```
model_path_pzza =
"/content/drive/MyDrive/stock_models/pzza_model.keras"
scaler_path_pzza =
"/content/drive/MyDrive/stock_models/pzza_scaler.pkl"

model_pzza = load_model(model_path_pzza)

with open(scaler_path_pzza, "rb") as f:
    pzza_scaler_saved = pickle.load(f)

df_pzza_fore = yf.download("PZZA", start="2025-03-31", end="2026-05-
01")
#2025-03-31= the first date of train dataset

/tmp/ipykernel_3310/3432172003.py:3: FutureWarning:
YF.download() has changed argument auto_adjust default to True

[*****100%*****] 1 of 1 completed

# Drop the Ticker level (level 1)
df_pzza_fore.columns = df_pzza_fore.columns.droplevel(1)

# (optional) ensure clean column names
df_pzza_fore.columns.name = None

# Final dataframe
df_pzza_final = df_pzza_fore[['Close', 'High', 'Low', 'Open',
'Volume']]

df_pzza_final = df_pzza_final.reset_index()

df_pzza_filtered = df_pzza_final[['Date', 'Close']]
df_pzza_filtered.set_index('Date', inplace=True)

pzza_data_values = df_pzza_filtered.values
pzza_data_values

array([[39.24121857],
       [39.70928192],
       [39.99585342],
       [39.17434692],
       [34.70383453],
       [33.57665253],
       [30.70137787],
       [32.34439087],
```

[30.99750519],
[30.44346619],
[29.62195969],
[29.08702469],
[29.65061569],
[29.76524353],
[29.49777794],
[30.17599869],
[31.27452278],
[31.70438004],
[31.98139954],
[32.20110321],
[32.21065903],
[32.98439789],
[33.05126953],
[34.09247589],
[33.8441124],
[32.30617905],
[31.80945587],
[36.87221909],
[36.55699539],
[39.21256256],
[39.59465408],
[38.46747208],
[38.2573204],
[38.5343399],
[39.87743759],
[39.41363144],
[39.61654663],
[41.0949173],
[40.82436752],
[42.8148613],
[42.11915207],
[42.3413887],
[43.72314072],
[44.74737167],
[44.82466888],
[45.50105286],
[45.90687561],
[48.22589874],
[48.82497787],
[46.57360077],
[50.04246521],
[48.92160416],
[48.71868515],
[48.11960602],
[47.44322968],
[47.32727814],
[47.87804794],

[47.34660721],
[47.38525391],
[46.23540497],
[46.89246368],
[47.04706192],
[47.28863144],
[47.81040573],
[48.19691467],
[47.34660721],
[45.94552994],
[46.67988968],
[46.31270981],
[46.39967346],
[45.43341446],
[45.31746674],
[42.54430008],
[42.58295822],
[42.57329178],
[43.53955078],
[43.23034668],
[43.819767],
[43.99369049],
[41.68433762],
[42.45734024],
[41.60703659],
[40.78571701],
[41.96455002],
[40.97896576],
[40.37989044],
[38.72759247],
[38.81454849],
[39.15274429],
[42.51531982],
[41.85826111],
[42.6505928],
[44.56378174],
[45.93586731],
[45.99384308],
[45.96485519],
[44.75499725],
[44.78427124],
[43.95492935],
[43.24267578],
[45.16479111],
[47.04787445],
[47.47718048],
[47.37960815],
[46.51124191],
[47.52596283],

[46.46245575],
[47.04787445],
[46.21853638],
[47.06739044],
[48.48213959],
[47.57474899],
[48.53092575],
[48.83338928],
[49.18464279],
[48.41384125],
[47.47718048],
[45.00867844],
[44.64767075],
[46.35512924],
[45.08673096],
[44.7159729],
[45.14527512],
[45.6136055],
[45.85752869],
[46.7551651],
[46.97957611],
[47.06739044],
[46.7551651],
[46.1892662],
[45.43798065],
[43.4573288],
[42.17917252],
[41.30104828],
[40.57903671],
[44.51107407],
[47.49669266],
[51.95560455],
[51.74095535],
[50.74574661],
[52.03366089],
[51.37994766],
[51.59460068],
[51.63362503],
[53.96553421],
[50.33596039],
[50.65793228],
[51.18480682],
[50.01397705],
[49.57491684],
[47.11617661],
[42.41333771],
[40.25705338],
[41.77913666],
[40.08143616],

[42.90118408],
[41.22299194],
[39.769207],
[39.72042465],
[40.10094452],
[39.68651962],
[40.48577118],
[38.82805634],
[38.03866577],
[39.71611786],
[39.93320084],
[40.65351868],
[41.30476379],
[41.51197815],
[41.03834534],
[41.58105087],
[42.3013649],
[41.79813385],
[40.614048],
[40.6041832],
[40.41670227],
[40.98900986],
[41.38370132],
[40.86073303],
[41.00874329],
[40.36736298],
[40.42657089],
[40.22922134],
[40.3180275],
[39.68651962],
[38.4629631],
[38.75898361],
[38.77872086],
[38.79845428],
[38.48270035],
[37.97946548],
[39.76545334],
[39.34115982],
[38.96620178],
[37.05193329],
[37.27888107],
[36.79538345],
[37.61437225],
[37.02233124],
[37.08153534],
[37.4071579],
[35.69023514],
[37.55516815],
[37.23941422],

[37.07166672],
[36.41055679],
[35.89745331],
[35.79877472],
[35.23633575],
[35.40408325],
[34.70349884],
[33.44047928],
[33.49968338],
[33.85490799],
[33.77597046],
[34.22000122],
[34.84000015],
[34.81999969],
[34.00999832],
[32.02000046],
[31.79000092],
[31.32999992],
[32.83000183],
[32.58000183],
[31.59000015],
[32.75999832],
[34.54000092],
[33.83000183],
[30.90999985],
[31.35000038],
[31.94000053],
[31.98999977],
[30.80999947],
[31.01000023],
[30.70000076],
[32.70999908],
[32.54000092],
[38.86000061],
[36.11999893],
[36.79000092],
[35.70000076],
[35.54000092],
[34.33000183],
[33.70000076],
[34.40999985],
[34.02999878],
[34.58000183],
[33.11999893],
[32.63999939],
[31.76000023],
[32.38999939],
[32.40999985],
[32.81000137],

```

[34.99000168],
[35.22999954],
[34.90000153],
[35.36999893],
[35.52999878],
[35.63000107],
[35.00999832],
[34.99000168],
[36.93999863],
[36.90000153],
[37.45999908],
[37.90999985],
[37.56999969],
[37.97000122],
[36.93000031],
[37.00999832],
[37.33000183],
[36.56000137],
[35.02999878],
[36.09500122]])

pizza_scaled_forecast_data =
pizza_scaler_saved.transform(pzza_data_values)

# Take the last sequence_dpz days from the dataset for generating
future predictions
pizza_last_days = pzza_scaled_forecast_data[-sequence_dpz:]

# Reshape
pizza_x_future = pzza_last_days.reshape((1, pzza_last_days.shape[0],
1))

# to store predictions for the next 15 days
pizza_future_predictions = []

for _ in range(15):
    pzza_pred = model_pzza.predict(pzza_x_future)
    pizza_future_predictions.append(pzza_pred[0, 0])

    pzza_x_future = np.append(pzza_x_future[:, 1:, :],
[[pzza_pred[0]]], axis=1)

1/1 _____ 0s 440ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 69ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 66ms/step

```

```
1/1 _____ 0s 66ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 57ms/step
```

```
# Convert back to the original scale
```

```
pzza_future_predictions =
pzza_scaler.inverse_transform(np.array(pzza_future_predictions).reshap
e(-1, 1))
```

```
pzza_future_predictions
```

```
array([[35.544765],
       [35.45854 ],
       [35.348682],
       [35.241005],
       [35.13347 ],
       [35.024284],
       [34.912983],
       [34.79991 ],
       [34.685787],
       [34.571415],
       [34.4575  ],
       [34.344624],
       [34.23316 ],
       [34.123383],
       [34.015392]], dtype=float32)
```

```
# Store in a df
```

```
pzza_forecast_dates = pd.date_range(start=df_pzza_filtered.index[-1] +
pd.Timedelta(days=1), periods=15, freq='B')
pzza_forecast = pd.DataFrame(pzza_future_predictions,
index=pzza_forecast_dates, columns=['Prediction'])
```

```
pzza_forecast
```

```
{"summary": "{\n  \"name\": \"pzza_forecast\",\n  \"rows\": 15,\n  \"fields\": [\n    {\n      \"column\": \"Prediction\",\n      \"properties\": {\n        \"dtype\": \"float32\",\n        \"num_unique_values\": 15,\n        \"samples\": [\n          34.571414947509766,\n          34.34462356567383,\n          35.54476547241211\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"pzza_forecast\"}
```

```
# visualize the prediction
```

```
pzza_fig = go.Figure()
```

```
# test data
```

```

pizza_fig.add_trace(go.Scatter(x=df_pzza_filtered.index,
y=df_pzza_filtered['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue')))

# future data
pizza_fig.add_trace(go.Scatter(x=pzza_forecast.index,
y=pzza_forecast['Prediction'], mode='lines', name='Predictions',
line=dict(color='red')))

# Update the layout
pizza_fig.update_layout(
    title="PZZA Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

pizza_fig.show()

```

MCD Modeling

```

# 30-day rolling mean and std
rolling_mean = pivot_df['MCD'].rolling(window=30).mean()
rolling_std = pivot_df['MCD'].rolling(window=30).std()

plt.figure(figsize=(14,6))

# MCD actual price
plt.plot(pivot_df['MCD'], label='MCD Close')

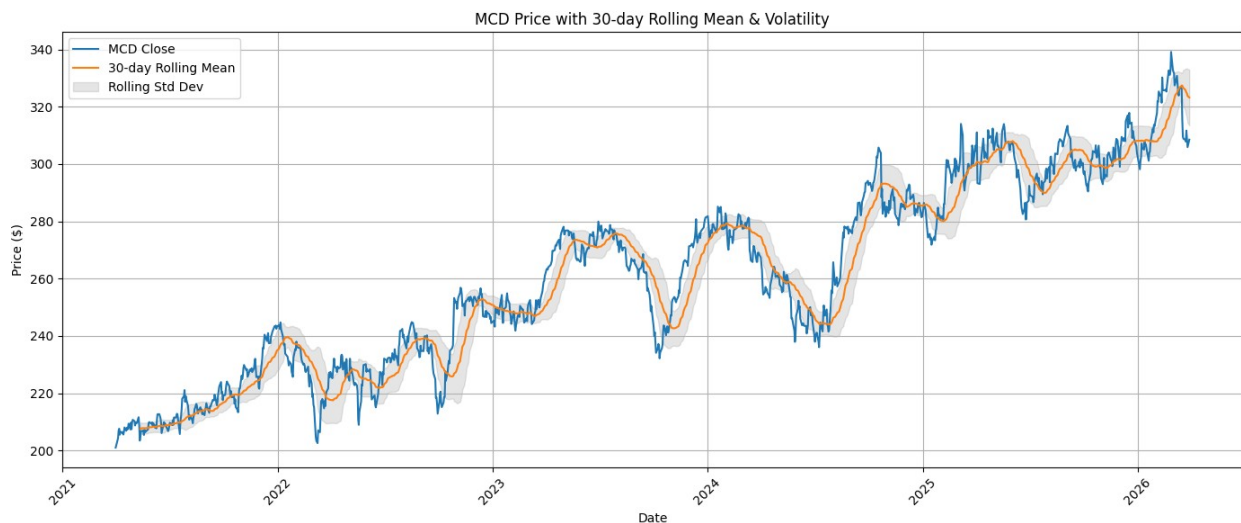
# rolling mean
plt.plot(rolling_mean, label='30-day Rolling Mean')

# volatility band
plt.fill_between(
    pivot_df.index,
    rolling_mean - rolling_std,
    rolling_mean + rolling_std,
    color='gray',
    alpha=0.2,
    label='Rolling Std Dev'
)

plt.title("MCD Price with 30-day Rolling Mean & Volatility")
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)

```

```
plt.tight_layout()
plt.show()
```



```
# 1. Convert date
final_df['Date'] = pd.to_datetime(final_df['Date'])

# 2. Set index
df_plot = final_df.set_index('Date')

ticker = 'MCD'
ticker_data = df_plot[df_plot['Company'] == ticker].copy()

# 4. Prepare OHLCV data
plot_data = ticker_data[['Open', 'High', 'Low', 'Close', 'Volume']]
plot_data = plot_data.sort_index().dropna()

# 5. Style
my_style = mpf.make_mpf_style(
    base_mpf_style='charles',
    gridstyle='--',
    facecolor='white'
)

# 6. Plot
fig, axlist = mpf.plot(
    plot_data,
    type='candle',
    style=my_style,
    volume=True,
    ylabel='Price ($)',
    ylabel_lower='Volume',
    figsize=(12, 6),
    returnfig=True,
```



```
    tight_layout=True
)

# 7. Title
axlist[0].set_title(
    f"{ticker} Stock Price Over Time",
    fontsize=12,
    pad=12
)

plt.show()

/usr/local/lib/python3.12/dist-packages/mplfinance/
_arg_validators.py:84: UserWarning:
```

```
=====

WARNING: YOU ARE PLOTTING SO MUCH DATA THAT IT MAY NOT BE
        POSSIBLE TO SEE DETAILS (Candles, Ohlc-Bars, Etc.)
For more information see:
- https://github.com/matplotlib/mplfinance/wiki/Plotting-Too-Much-Data

TO SILENCE THIS WARNING, set `type='line'` in `mpf.plot()`
OR set kwarg `warn_too_much_data=N` where N is an integer
LARGER than the number of data points you want to plot.

=====
```



Train Test Data Preparation

```
seed = 42
np.random.seed(seed)

random.seed(seed)
tf.random.set_seed(seed)
os.environ['PYTHONHASHSEED'] = str(seed)

# DATA PREPARATION
mcd_df_filtered = mcd_df[['Date', 'Close']]
mcd_df_filtered.set_index('Date', inplace=True)

mcd_df_filtered.index = pd.to_datetime(mcd_df_filtered.index)
mcd_df_filtered = mcd_df_filtered.sort_index()

mcd_train_size = int(len(mcd_df_filtered) * 0.8)

mcd_train_data = mcd_df_filtered[:mcd_train_size]
mcd_test_data = mcd_df_filtered[mcd_train_size:]

mcd_train_data_values = mcd_train_data.values
mcd_test_data_values = mcd_test_data.values

# SCALING
mcd_scaler = StandardScaler()
mcd_scaled_train_data =
mcd_scaler.fit_transform(mcd_train_data_values)
mcd_scaled_test_data = mcd_scaler.transform(mcd_test_data_values)
```

```

# SEQUENCE
sequence_mcd = 90

# TRAIN DATA
mcd_X_train, mcd_y_train = [], []

for i in range(sequence_mcd, len(mcd_scaled_train_data)):
    mcd_X_train.append(mcd_scaled_train_data[i-sequence_mcd:i, 0])
    mcd_y_train.append(mcd_scaled_train_data[i, 0])

mcd_X_train, mcd_y_train = np.array(mcd_X_train),
np.array(mcd_y_train)

mcd_X_train = np.reshape(mcd_X_train, (mcd_X_train.shape[0],
mcd_X_train.shape[1], 1))

# TEST DATA
mcd_last_points = mcd_scaled_train_data[-sequence_mcd:]
mcd_combined_test_data = np.concatenate((mcd_last_points,
mcd_scaled_test_data), axis=0)

mcd_X_test, mcd_y_test = [], mcd_test_data_values

for i in range(sequence_mcd, len(mcd_combined_test_data)):
    mcd_X_test.append(mcd_combined_test_data[i-sequence_mcd:i, 0])

mcd_X_test = np.array(mcd_X_test)
mcd_X_test = np.reshape(mcd_X_test, (mcd_X_test.shape[0],
mcd_X_test.shape[1], 1))

```

Model Development

```

mcd_model = keras.models.Sequential()

mcd_model.add(keras.layers.LSTM(128, return_sequences=True,
input_shape=(mcd_X_train.shape[1], 1)))
mcd_model.add(keras.layers.LSTM(256, return_sequences=False))
mcd_model.add(keras.layers.Dense(512, activation="relu"))
mcd_model.add(keras.layers.Dropout(0.3))
mcd_model.add(keras.layers.Dense(1))

mcd_model.compile(optimizer="adam",
                  loss="mae",
                  metrics=['mse'])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/
rnn.py:199: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the

```

first layer in the model instead.

```
mcd_model.summary()
```

Model: "sequential_5"

Layer (type) Param #	Output Shape
lstm_10 (LSTM) 66,560	(None, 90, 128)
lstm_11 (LSTM) 394,240	(None, 256)
dense_10 (Dense) 131,584	(None, 512)
dropout_5 (Dropout) 0	(None, 512)
dense_11 (Dense) 513	(None, 1)

Total params: 592,897 (2.26 MB)

Trainable params: 592,897 (2.26 MB)

Non-trainable params: 0 (0.00 B)

```
mcd_first_callback = keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

Training the Model

```
mcd_training = mcd_model.fit(  
    mcd_X_train,  
    mcd_y_train,
```

```
epochs=50,  
batch_size=32,  
validation_split=0.2,  
callbacks=[mcd_first_callback],  
shuffle=False  
)  
  
Epoch 1/50  
23/23 _____ 16s 543ms/step - loss: 0.3362 - mse: 0.2112  
- val_loss: 0.8488 - val_mse: 0.8627  
Epoch 2/50  
23/23 _____ 20s 533ms/step - loss: 0.3280 - mse: 0.1661  
- val_loss: 0.2742 - val_mse: 0.1182  
Epoch 3/50  
23/23 _____ 12s 540ms/step - loss: 0.2457 - mse: 0.1127  
- val_loss: 0.6082 - val_mse: 0.4486  
Epoch 4/50  
23/23 _____ 20s 512ms/step - loss: 0.3207 - mse: 0.1482  
- val_loss: 0.1615 - val_mse: 0.0442  
Epoch 5/50  
23/23 _____ 13s 563ms/step - loss: 0.1801 - mse: 0.0553  
- val_loss: 0.1995 - val_mse: 0.0606  
Epoch 6/50  
23/23 _____ 12s 528ms/step - loss: 0.1496 - mse: 0.0365  
- val_loss: 0.2460 - val_mse: 0.0879  
Epoch 7/50  
23/23 _____ 11s 493ms/step - loss: 0.1703 - mse: 0.0474  
- val_loss: 0.2699 - val_mse: 0.1032  
Epoch 8/50  
23/23 _____ 12s 490ms/step - loss: 0.1557 - mse: 0.0384  
- val_loss: 0.1590 - val_mse: 0.0401  
Epoch 9/50  
23/23 _____ 21s 536ms/step - loss: 0.1500 - mse: 0.0359  
- val_loss: 0.1559 - val_mse: 0.0387  
Epoch 10/50  
23/23 _____ 12s 515ms/step - loss: 0.1509 - mse: 0.0372  
- val_loss: 0.1669 - val_mse: 0.0440  
Epoch 11/50  
23/23 _____ 10s 433ms/step - loss: 0.1361 - mse: 0.0303  
- val_loss: 0.1964 - val_mse: 0.0582  
Epoch 12/50  
23/23 _____ 13s 565ms/step - loss: 0.1465 - mse: 0.0333  
- val_loss: 0.1536 - val_mse: 0.0381  
Epoch 13/50  
23/23 _____ 12s 537ms/step - loss: 0.1288 - mse: 0.0266  
- val_loss: 0.1567 - val_mse: 0.0394  
Epoch 14/50  
23/23 _____ 13s 567ms/step - loss: 0.1317 - mse: 0.0277  
- val_loss: 0.1422 - val_mse: 0.0336  
Epoch 15/50
```

```
23/23 _____ 11s 493ms/step - loss: 0.1248 - mse: 0.0254
- val_loss: 0.1559 - val_mse: 0.0387
Epoch 16/50
23/23 _____ 12s 491ms/step - loss: 0.1227 - mse: 0.0252
- val_loss: 0.1420 - val_mse: 0.0332
Epoch 17/50
23/23 _____ 13s 572ms/step - loss: 0.1248 - mse: 0.0259
- val_loss: 0.1305 - val_mse: 0.0290
Epoch 18/50
23/23 _____ 12s 546ms/step - loss: 0.1138 - mse: 0.0211
- val_loss: 0.1232 - val_mse: 0.0265
Epoch 19/50
23/23 _____ 13s 569ms/step - loss: 0.1185 - mse: 0.0228
- val_loss: 0.1085 - val_mse: 0.0218
Epoch 20/50
23/23 _____ 13s 579ms/step - loss: 0.1499 - mse: 0.0362
- val_loss: 0.1575 - val_mse: 0.0384
Epoch 21/50
23/23 _____ 11s 476ms/step - loss: 0.1157 - mse: 0.0222
- val_loss: 0.1085 - val_mse: 0.0217
Epoch 22/50
23/23 _____ 22s 547ms/step - loss: 0.1275 - mse: 0.0266
- val_loss: 0.1065 - val_mse: 0.0211
Epoch 23/50
23/23 _____ 13s 564ms/step - loss: 0.1538 - mse: 0.0387
- val_loss: 0.1122 - val_mse: 0.0224
Epoch 24/50
23/23 _____ 12s 520ms/step - loss: 0.1199 - mse: 0.0239
- val_loss: 0.1076 - val_mse: 0.0218
Epoch 25/50
23/23 _____ 10s 437ms/step - loss: 0.1208 - mse: 0.0236
- val_loss: 0.1004 - val_mse: 0.0193
Epoch 26/50
23/23 _____ 23s 567ms/step - loss: 0.1753 - mse: 0.0508
- val_loss: 0.1248 - val_mse: 0.0262
Epoch 27/50
23/23 _____ 18s 447ms/step - loss: 0.1126 - mse: 0.0210
- val_loss: 0.1009 - val_mse: 0.0196
Epoch 28/50
23/23 _____ 22s 539ms/step - loss: 0.1371 - mse: 0.0309
- val_loss: 0.1642 - val_mse: 0.0401
Epoch 29/50
23/23 _____ 20s 482ms/step - loss: 0.1298 - mse: 0.0276
- val_loss: 0.1502 - val_mse: 0.0340
Epoch 30/50
23/23 _____ 12s 539ms/step - loss: 0.1462 - mse: 0.0341
- val_loss: 0.2464 - val_mse: 0.0798
```

```
pd.DataFrame(mcd_training.history).plot()
```

Model Evaluation

```
# PREDICTION
mcd_predictions = mcd_model.predict(mcd_X_test)
mcd_predictions = mcd_scaler.inverse_transform(mcd_predictions)

mcd_test = mcd_test_data
mcd_test['mcd_predictions'] = mcd_predictions
mcd_test.plot()

mcd_fig = go.Figure()

mcd_fig.add_trace(go.Scatter(x=mcd_train_data.index,
                             y=mcd_train_data['Close'], mode='lines', name='Train (Actual)',
                             line=dict(color='blue')))
mcd_fig.add_trace(go.Scatter(x=mcd_test.index, y=mcd_test['Close'],
                             mode='lines', name='Test (Actual)', line=dict(color='orange')))
mcd_fig.add_trace(go.Scatter(x=mcd_test.index,
                             y=mcd_test['mcd_predictions'], mode='lines', name='Predictions',
                             line=dict(color='red')))

mcd_fig.update_layout(
    title="Stock Predictions MCD",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

mcd_fig.show()

#mae
mcd_mae = mean_absolute_error(mcd_y_test, mcd_predictions)
print(f'MAE: {mcd_mae}')

#R2
mcd_r2 = r2_score(mcd_y_test, mcd_predictions)
print(f'R²: {mcd_r2}')
```

Model Predictions

```
# FULL PREDICTION
mcd_train_predict = mcd_model.predict(mcd_X_train)
mcd_test_predict = mcd_model.predict(mcd_X_test)
mcd_train_predict = mcd_scaler.inverse_transform(mcd_train_predict)
mcd_test_predict = mcd_scaler.inverse_transform(mcd_test_predict)

mcd_full_predictions = pd.DataFrame(np.concatenate((mcd_train_predict,
mcd_test_predict), axis=0))

mcd_filtered_data_after_seq =
mcd_df_filtered.iloc[sequence_mcd:].copy()
mcd_full_predictions.index = mcd_filtered_data_after_seq.index
```

```

mcd_full_predictions.columns = ['Predictions']

mcd_fig = go.Figure()

mcd_fig.add_trace(go.Scatter(x=mcd_df_filtered.index,
                             y=mcd_df_filtered['Close'], mode='lines', name='Actual',
                             line=dict(color='blue'))))
mcd_fig.add_trace(go.Scatter(x=mcd_full_predictions.index,
                             y=mcd_full_predictions['Predictions'], mode='lines',
                             name='Predictions', line=dict(color='red'))))

mcd_fig.update_layout(
    title="Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

mcd_fig.show()

```

Future Forecast

```

model_path_mcd = "/content/drive/MyDrive/stock_models/mcd_model.keras"
scaler_path_mcd = "/content/drive/MyDrive/stock_models/mcd_scaler.pkl"

model_mcd = load_model(model_path_mcd)

with open(scaler_path_mcd, "rb") as f:
    mcd_scaler_saved = pickle.load(f)

df_mcd_fore = yf.download("MCD", start="2025-03-31", end="2026-05-01")
#2025-03-31= the first date of train dataset

/tmp/ipykernel_3310/2022883329.py:1: FutureWarning:
YF.download() has changed argument auto_adjust default to True

[*****100%*****] 1 of 1 completed

# Drop the Ticker level (level 1)
df_mcd_fore.columns = df_mcd_fore.columns.droplevel(1)

# (optional) ensure clean column names
df_mcd_fore.columns.name = None

# Final dataframe
df_mcd_final = df_mcd_fore[['Close', 'High', 'Low', 'Open', 'Volume']]

df_mcd_final = df_mcd_final.reset_index()

```



```
df_mcd_filtered = df_mcd_final[['Date', 'Close']]
df_mcd_filtered.set_index('Date', inplace=True)
```

```
mcd_data_values = df_mcd_filtered.values
mcd_data_values
```

```
array([[305.30706787],
       [306.53857422],
       [304.53491211],
       [311.07366943],
       [293.32424927],
       [293.10925293],
       [293.05059814],
       [297.08721924],
       [299.84344482],
       [302.8928833 ],
       [308.9234314 ],
       [305.41461182],
       [302.11102295],
       [304.26126099],
       [301.73959351],
       [311.86532593],
       [311.27890015],
       [309.07003784],
       [309.57824707],
       [309.64666748],
       [307.3888855 ],
       [312.42245483],
       [306.54833984],
       [304.90631104],
       [308.58132935],
       [309.62716675],
       [310.93682861],
       [306.40170288],
       [306.58743286],
       [304.86727905],
       [301.71026611],
       [300.54718018],
       [307.36935425],
       [311.40594482],
       [313.96673584],
       [312.56903076],
       [309.28500366],
       [307.20318604],
       [307.45733643],
       [307.82873535],
       [305.76641846],
       [304.80859375],
       [306.75363159],
       [307.34335327],
```

[307.05831909],
[306.25228882],
[303.70648193],
[302.11413574],
[299.57818604],
[295.3024292],
[296.57040405],
[297.98587036],
[296.75720215],
[291.23312378],
[287.3210144],
[284.68676758],
[282.55380249],
[284.5786438],
[286.49536133],
[280.67636108],
[280.7550354],
[286.57397461],
[287.18344116],
[292.41262817],
[289.60140991],
[289.0607605],
[288.52017212],
[286.69195557],
[288.01885986],
[293.29727173],
[294.79129028],
[296.72769165],
[294.50628662],
[293.79855347],
[295.24346924],
[291.99975586],
[290.69247437],
[294.06396484],
[293.03182983],
[289.45397949],
[293.37588501],
[294.11309814],
[297.50421143],
[298.4281311],
[294.94857788],
[297.72045898],
[299.03759766],
[293.67077637],
[302.4090271],
[302.65475464],
[300.23675537],
[299.16534424],
[296.49179077],

[299.86322021],
[303.67703247],
[303.65731812],
[303.43130493],
[305.62319946],
[307.73654175],
[307.87414551],
[308.70965576],
[308.09039307],
[307.59893799],
[306.11468506],
[306.89123535],
[308.18869019],
[312.13284302],
[312.75561523],
[313.34869385],
[310.76870728],
[309.20684814],
[308.93005371],
[302.42565918],
[304.88702393],
[301.8918457],
[298.60009766],
[299.80609131],
[301.46676636],
[297.68081665],
[298.90655518],
[299.50952148],
[299.70724487],
[300.83410645],
[298.82745361],
[301.73370361],
[299.50952148],
[300.39920044],
[297.06790161],
[296.43527222],
[297.52261353],
[292.63937378],
[292.91616821],
[291.01818848],
[290.43499756],
[293.59823608],
[296.66259766],
[301.06149292],
[301.74359131],
[301.82263184],
[304.55093384],
[304.23461914],
[303.94796753],

[306.54772949],
[303.44381714],
[302.27737427],
[306.43899536],
[302.88037109],
[298.87689209],
[298.95596313],
[295.00192261],
[292.96557617],
[295.77294922],
[302.15875244],
[294.98214722],
[296.21777344],
[295.66424561],
[303.30541992],
[303.41415405],
[304.04678345],
[303.50314331],
[301.39758301],
[301.09115601],
[299.26239014],
[300.66610718],
[305.79647827],
[301.39758301],
[306.8838501],
[308.81143188],
[308.23809814],
[301.88360596],
[299.04943848],
[306.00061035],
[306.82598877],
[309.50106812],
[308.06906128],
[309.06350708],
[308.80493164],
[307.98947144],
[314.96054077],
[316.95941162],
[312.75289917],
[316.9196167],
[317.87426758],
[314.08544922],
[314.46331787],
[309.11322021],
[311.58938599],
[308.95410156],
[306.81604004],
[306.31881714],
[303.93215942],

[301.5753479],
[298.19421387],
[301.08804321],
[302.47033691],
[307.16412354],
[305.61279297],
[305.04592896],
[307.7210083],
[306.41827393],
[306.9055481],
[305.72216797],
[301.15765381],
[303.99182129],
[304.32992554],
[307.53204346],
[311.21151733],
[312.38494873],
[311.06231689],
[313.7572937],
[313.25012207],
[316.76049805],
[317.70523071],
[321.8918457],
[321.68301392],
[325.34255981],
[323.79122925],
[324.15917969],
[321.41448975],
[330.23519897],
[325.76022339],
[325.79998779],
[326.06851196],
[325.29281616],
[327.40106201],
[332.70144653],
[331.1998291],
[331.1600647],
[332.67160034],
[339.16534424],
[332.96002197],
[332.17001343],
[331.73999023],
[327.45001221],
[328.05999756],
[330.79998779],
[327.94000244],
[325.20999146],
[323.91000366],
[326.45999146],

```
[326.6499939 ],
[326.29998779],
[315.73001099],
[309.57998657],
[308.8500061 ],
[308.47000122],
[307.83999634],
[311.70001221],
[308.92999268],
[305.8999939 ],
[308.52999878],
[310.79000854],
[307.29000854],
[307.14001465],
[309.76000977],
[304.8500061 ],
[307.01000977],
[309.54998779],
[305.67999268],
[304.51000977],
[303.22000122],
[306.26000977],
[306.95999146],
[311.35998535],
[306.94000244],
[301.83999634],
[300.07000732],
[302.52999878],
[299.35998535],
[290.20999146],
[292.39001465],
[290.07998657],
[292.76000977]])
```

```
mcd_scaled_forecast_data = mcd_scaler_saved.transform(mcd_data_values)
```

```
# Take the last sequence_dpz days from the dataset for generating  
future predictions
```

```
mcd_last_days = mcd_scaled_forecast_data[-sequence_dpz:]
```

```
# Reshape
```

```
mcd_x_future = mcd_last_days.reshape((1, mcd_last_days.shape[0], 1))
```

```
# to store predictions for the next 15 days
```

```
mcd_future_predictions = []
```

```
for _ in range(15):
```

```
    mcd_pred = model_mcd.predict(mcd_x_future)
```

```
    mcd_future_predictions.append(mcd_pred[0, 0])
```

```
mcd_x_future = np.append(mcd_x_future[:, 1:, :], [[mcd_pred[0]]],  
axis=1)
```

```
1/1 _____ 1s 541ms/step  
1/1 _____ 0s 147ms/step  
1/1 _____ 0s 127ms/step  
1/1 _____ 0s 185ms/step  
1/1 _____ 0s 137ms/step  
1/1 _____ 0s 133ms/step  
1/1 _____ 0s 118ms/step  
1/1 _____ 0s 148ms/step  
1/1 _____ 0s 110ms/step  
1/1 _____ 0s 179ms/step  
1/1 _____ 0s 244ms/step  
1/1 _____ 0s 293ms/step  
1/1 _____ 0s 225ms/step  
1/1 _____ 0s 158ms/step  
1/1 _____ 0s 205ms/step
```

```
# Convert back to the original scale
```

```
mcd_future_predictions =  
mcd_scaler.inverse_transform(np.array(mcd_future_predictions).reshape(  
-1, 1))  
mcd_future_predictions
```

```
array([[292.05176],  
       [292.35287],  
       [292.60123],  
       [292.8028 ],  
       [292.94458],  
       [293.02606],  
       [293.05655],  
       [293.05072],  
       [293.0241 ],  
       [292.99057],  
       [292.96033],  
       [292.93954],  
       [292.93073],  
       [292.93292],  
       [292.94345]], dtype=float32)
```

```
# Store in a df
```

```
mcd_forecast_dates = pd.date_range(start=df_mcd_filtered.index[-1] +  
pd.Timedelta(days=1), periods=15, freq='B')  
mcd_forecast = pd.DataFrame(mcd_future_predictions,  
index=mcd_forecast_dates, columns=['Prediction'])  
  
mcd_forecast
```

```

{"summary": "{\n  \"name\": \"mcd_forecast\",\n  \"rows\": 15,\n  \"fields\": [\n    {\n      \"column\": \"Prediction\",\n      \"properties\": {\n        \"dtype\": \"float32\",\n        \"num_unique_values\": 15,\n        \"samples\": [\n          292.9905700683594,\n          292.9395446777344,\n          292.0517578125\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"mcd_forecast\"}

# visualize the prediction
mcd_fig = go.Figure()

# test data
mcd_fig.add_trace(go.Scatter(x=df_mcd_filtered.index,
y=df_mcd_filtered['Close'], mode='lines', name='Train (Actual)',
line=dict(color='blue')))

# future data
mcd_fig.add_trace(go.Scatter(x=mcd_forecast.index,
y=mcd_forecast['Prediction'], mode='lines', name='Predictions',
line=dict(color='red')))

# Update the layout
mcd_fig.update_layout(
    title="MCD Stock Predictions",
    xaxis_title="Date",
    yaxis_title="Close Price"
)

mcd_fig.show()

```

Save models

```

import os
import pickle

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

save_dir = "/content/drive/MyDrive/stock_models"
os.makedirs(save_dir, exist_ok=True)

dpz_model_path = os.path.join(save_dir, "dpz_model.keras")
pzza_model_path = os.path.join(save_dir, "pzza_model.keras")
mcd_model_path = os.path.join(save_dir, "mcd_model.keras")

dpz_scaler_path = os.path.join(save_dir, "dpz_scaler.pkl")

```



```
pzza_scaler_path = os.path.join(save_dir, "pzza_scaler.pkl")
mcd_scaler_path = os.path.join(save_dir, "mcd_scaler.pkl")

# SAVE MODELS
dpz_model.save(dpz_model_path)
pzza_model.save(pzza_model_path)
mcd_model.save(mcd_model_path)

# SAVE SCALERS (IMPORTANT)
with open(dpz_scaler_path, "wb") as f:
    pickle.dump(dpz_scaler, f)

with open(pzza_scaler_path, "wb") as f:
    pickle.dump(pzza_scaler, f)

with open(mcd_scaler_path, "wb") as f:
    pickle.dump(mcd_scaler, f)
```